COMP101
Introduction to Programming
2025-26

Lecture-06

String and Number Output Control

Output Control - String and Number

- There are several techniques to display output as we want it
  - Concatentation – joins strings together (maybe to make a new word)
  - Comma control – spaces strings apart
  - Separator control – puts our own chosen character in between words
  - String methods – to change between upper and lower case
  - Functions – gives information about a string, e.g. its length etc

- A good ‘rule of thumb’ is to keep it simple – no fancy output – keep it clear and accurate. It is the user who has to understand it

Concatenation (+) of strings

- Only works with strings – use the '+' operator
- Joins string vars with string vars or string vars with string literals

```
print ("Hello World")
Hello World
```

This is a 'string literal' in the parentheses - it's not been typed in by the user

```
words = input ("Enter two words: ") Hello World
print (words)
Hello World
```

This is a 'string var' - it's been typed in by the user

Concatenation (+) of strings

```
word1 = “Bat”          #code line1: ‘string literal’ – has a fixed value: would be a ‘string var’ if input by user
word2 = “man”          #code line2: as above
print (word1,word2) #code line3:comma puts a space between the words in the output string
Bat man

print (word1 + word2) #code line4: concatenate (‘join’) word1 (Bat) with word2 (man) – no space in output
Batman

print (word1 + word2”and Robin”)  #code line5:  Concatenates and outputs a further string
Batmanand Robin

print (word1 + word2,”and Robin”) #code line6: the comma after the concatenation outputs a space
Batman and Robin
```

Concatenation – number with string

```
num1 = int ( input("Enter a number: ") ) 21    #21 is cast as integer
print ("Number entered is"+(num1))
TypeError
```

This can't be done: Can't concatenate int with a string:

num1 is integer (21) - Python reads the '+' as plus

#two solutions

```
print ("Number entered is“,+(num1))
Number entered is 21
```

Solution-1: Comma before the concatenation.

Also puts a space after the string

```
print ("Number entered is "+str(num1))
Number entered is 21
```

Solution-2: cast the integer to a str

No comma before '+' as it gives an error.

No comma means we need put a space inside the quotes after the word 'is'

Some internal programming is in charge here – done by the developer

Comma output control

```
user_name = input (“Enter your name: “) John
print (“Hello”,user_name)
Hello John
```

One space here

We use a comma to separate the literal (”Hello”) from the variable (user_name) on output.
The variable does not go in quotes and the comma will output a single space between the words

```
user_name = input (“Enter your name: “) Jean
print (“Hello ”,user_name)
Hello  Jean
```

Two spaces here

If we put a space in the literal (“Hello “), we still need a comma to separate the literal (”Hello ”) from the variable (user_name). We now get two spaces between the words – one from the string-literal and the other from the comma

Separator control - sep=(“”)

- Output is usually done with a default separator – a comma outputs a space
- We can customise the separator to something else
- sep=“” redefines the separator – substitutes the comma with a user-selected character

```
print (10,20,30)            #normal output, comma puts a space between values
10 20 30
print (10,20,30,sep="")     #sep=”” – no space between the quotes gives no space in output
102030
print (10,20,30,sep=".")    #where there’s a comma to output a space, put a full stop
10.20.30
print (10,20,30,sep="/")    #where there’s a comma to output a space, put a forward slash
10/20/30
```

Separator with variables

```
name = (“Mary”)
print (“Hello”,name)
Hello Mary
```

comma must be used to separate the literal (“Hello”) from the variable (name):

comma must be used to separate the variable (name) from the sep= operator:

```
print ("Hello", name, sep="/")
Hello/Mary
```

‘/’ will replace the space

```
print (“Hello", name, sep="")
HelloMary
```

A sep with no space in the quotes gives no space in the output between the literal and the variable

The use of ‘sep’ applies to numbers, strings and vars – no difference

String Methods

- String methods can be used to control output
- The methods are ‘called’ using dot notation

```
words = input (“Enter two words space separated: “) Cat dog
print (words)
Cat dog
#string methods
print (words.upper() )
CAT DOG
print (words.lower() )
cat dog
print (words.title() )
Cat Dog
```

Don’t forget the dot and the parentheses:
var_name.upper()
var_name.lower()
var_name.title()

Can be useful to ‘force’ user input to a format that can be checked more easily
e.g. password control

Built-in Functions

- Python has built-in functions ('standard functions')
  - These don't need dot notation

- len()
  - returns the length of a string argument - does not work on numbers
  - An argument is what the user types in

```
nums = input ("Enter some numbers: ") 74982
print (len(nums))
5
```

Remember – input is a string

If 3.14159 is input as string, then len = 7 as the decimal point counts as a character

len(nums)
len is the function
(nums) in parentheses is the 'argument'

String subscripts

- Strings are subscripted – starting at zero
- Spaces in a string count as a character

```
word = “Test One”
0    1    2    3    4    5    6    7
T    e    s    t         O    n    e
print (len(word))
8
print (len(word[2:5])) #can apply to substrings
3
print (word[2:7])
st On
```

Surprising answer: Python doesn’t include the start subscript in the range

… and here it doesn’t include the end subscript in the range

String and Number I/O – what goes wrong

```
job = input (“Enter job role: “) cleaner
print (Job)                                  #error – var name

job = input(“Enter job role’) cleaner        #error – quotes
print (job)

job = input {“Enter job role: “} cleaner     #error - brackets
print (job)
```

String and Number I/O – what goes wrong

```
job = input (“Enter job role: “) cleaner
print (‘job’)                                   #error – o/p quotes

job = input (“Enter job role’ cleaner           #error – brackets
print (job)

while = input(“Enter job role: “) cleaner       #error – reserved word
print (while)
```